Assignment 4 Report

Mohamad Osman
301590514

Overview

For this assignment, I reviewed my teammate Keith's `Interface.java` class, which handles the game's UI, including menus, buttons, and game states. While the code was functional, I identified several code smells that affected readability and maintainability.

Work Distribution

I identified eight code smells in my teammate's code. These were originally meant for my teammate to refactor, but due to a misunderstanding, I refactored four of them myself. I then provided four new code smells for my teammate to complete.

Additionally, my teammate Devaditya identified four code smells in my `DiamondSpawner` class, which I refactored myself. Overall, I identified eight code smells and refactored eight in total.

Code Smells I Assigned to kieth

In addition to the ones I refactored, I identified four additional code smells and assigned them to my teammate to complete. One issue was that `Interface.java` had become a large class, handling too many responsibilities, and could be improved by separating rendering and control logic. Another issue was the overuse of switch statements for handling game states, which could be improved using a State pattern or separate handlers.There was also tight coupling between the UI and the `Board` class, where UI logic directly accessed game logic, reducing flexibility. This could be improved by introducing an abstraction such as a `GameManager`. Lastly, font objects were being created repeatedly in

multiple places, which could be simplified by using shared constants instead.

Keith's s code that I refactored

The `paintComponent()` method was too long and handled multiple responsibilities. I fixed this by breaking it into smaller methods like `drawCurrentScreen()` and `drawButtons()`.

There was also duplication in the `buildButtons()` method, which I resolved by creating a helper method `addButton()`.

The class used many hardcoded values, so I created a `UIConstants` class to store them.

Finally, there was an unused method `btn()`, which I removed to clean up the code.

Devaditya code smells(for me) that i refactored

In my own `DiamondSpawner` class, I replaced the hardcoded timer value with a constant `DIAMOND_SPAWN_DELAY` to improve readability. I identified a feature envy issue, where the class relied too heavily on the `Board` for diamond logic, and also a violation of the Single Responsibility Principle, since it handled both timing and game logic. I addressed both issues by moving the diamond logic into the `Board` class through a new method `updateDiamondState()`, allowing the spawner to focus only on timing. Finally, I improved flexibility by adding an overloaded constructor that allows different delay values instead of using a fixed one.

Conclusion

By refactoring both my teammate’s code and my own, I improved the overall structure and readability of the project. These changes make the code easier to maintain and extend going forward.